



Ingeniería de Software



Implementación - Pruebas

Modelo de Proceso

► Modelo en Cascada

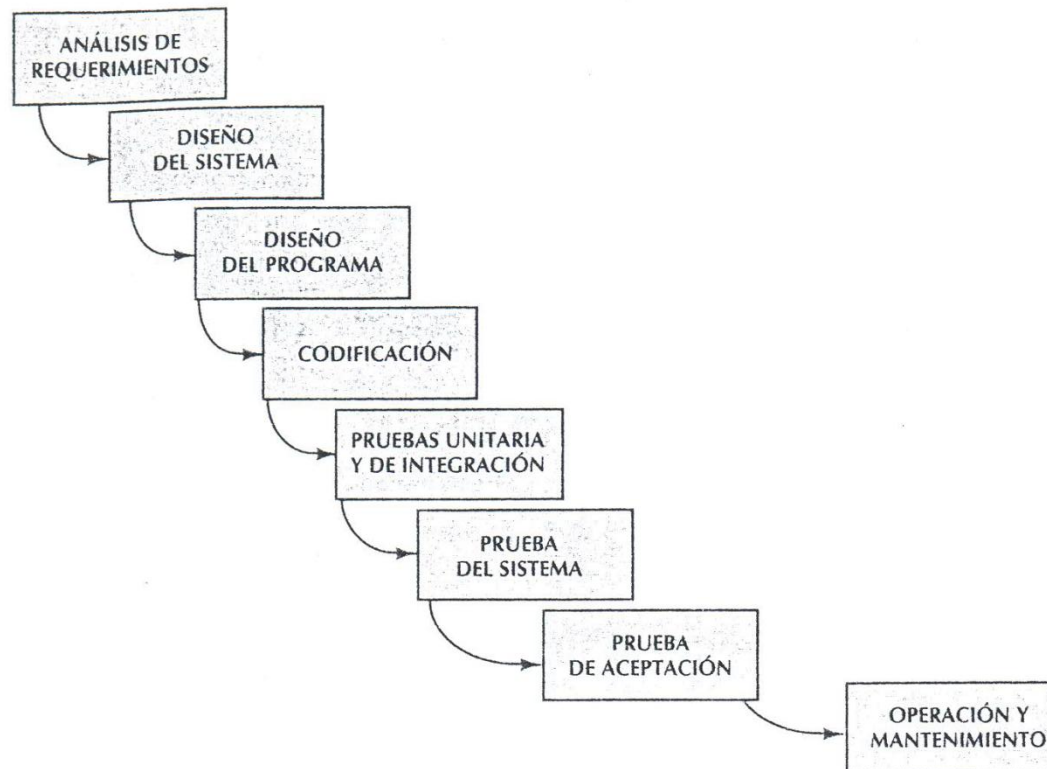


Figura 2.1. El modelo de cascada.

Codificación

- ▶ Una vez establecido el diseño, se debe escribir los programas que implementen dicho diseño.
- ▶ Esto puede resultar una tarea compleja por distintos motivos:
- ▶ Los diseñadores pueden no haber tenido en cuenta las particularidades de la plataforma y el ambiente de programación.
- ▶ Las estructuras e interrelaciones que son fáciles de describir mediante diagramas, no siempre resultan sencillas de escribir en código.
- ▶ Es indispensable escribir el código de forma que resulte comprensible para otras personas.
- ▶ Se debe sacar beneficios de las características de diseño creando código que sea reutilizable.

Codificación - Pautas Generales

- ▶ Resultan útiles para conservar la calidad del diseño en la codificación:
- ▶ Localización de entrada y salida: es deseable localizarlas en componentes separados del resto del código ya que generalmente son más difíciles de probar.
- ▶ Inclusión de pseudocódigo: Es útil avanzar el diseño, realizando un pseudocódigo para adaptar el diseño al lenguaje elegido.
- ▶ Revisión y reescritura, no a los remiendos: Es recomendable realizar un borrador, revisarlo y reescribirlo tantas veces como sea necesario.
- ▶ Reutilización: Hay dos clases de reutilización:
 - ▶ Productiva: se crean componentes destinados a ser reutilizados por otra aplicación
 - ▶ Consumidora: Se usan componentes originalmente desarrollados para otros proyectos.

Codificación: Documentación

- ▶ Se considera como Documentación del programa al conjunto de descripciones escritas que explican al lector qué hace el programa y cómo lo hace.
- ▶ Se divide en:
 - ▶ Documentación interna: Es concisa, escrita en un nivel apropiado para un programador. Contiene información dirigida a quienes leerán el código fuente. Incluye información de algoritmos, estructuras de control, flujos de control.
 - ▶ Documentación externa: Se prepara para ser leída por quienes, tal vez, nunca verán el código real. Por ejemplo, los diseñadores, cuando evalúan modificaciones o mejoras.



Ingeniería de Software



Prueba del software

Prueba del software

- ▶ La prueba es un elemento de un tema más amplio que se conoce como Verificación y Validación (V&V)
- ▶ V&V
 - ▶ Es un proceso que pretende establecer la existencia de defectos en un sistema de software.
 - ▶ Tienen lugar en cada etapa del proceso de software.
 - ▶ V&V comienza con revisiones de los requerimientos y continúa con revisiones del diseño e inspecciones de código hasta la prueba del producto.
 - ▶ V&V no son lo mismo, aunque a menudo se confunden.

V&V

- ▶ Validación
 - ▶ ¿Estamos construyendo el producto correcto?
- ▶ Verificación
 - ▶ ¿Estamos construyendo el producto correctamente ?

- ▶ Esto nos dice que la verificación implica comprobar que el software está de acuerdo con su especificación. Donde se debe comprobar que satisface tanto los requerimientos funcionales como los no funcionales.
- ▶ La validación sin embargo es un proceso más general, cuyo objetivo es asegurar que el software satisface las expectativas del cliente.
- ▶ V&V abarca una amplia lista de actividades.

Prueba del software

- ▶ La etapa de Prueba no es la primera instancia en que se localizan defectos; se ha visto que en la revisión de requerimientos y el diseño contribuyen a descubrir los problemas en las etapas tempranas.
- ▶ ¿Qué significa que el software ha fallado?
 - ▶ El software no hace lo que especifican los requerimientos
- ▶ Posibles razones:
 - ▶ Especificación errónea.
 - ▶ Requerimientos imposibles con las estructuras previstas.
 - ▶ Defectos en diseño del sistema.
 - ▶ Defectos en diseño del programa.
 - ▶ Defectos en código.

Tipos de Defectos

- ▶ **Algorítmicos**
 - ▶ Ej.: No inicializar variables
- ▶ **De sintaxis**
 - ▶ Ej.: Confundir un 0 por una O
- ▶ **De precisión**
 - ▶ Ej.: Fórmulas no implementadas correctamente
- ▶ **De documentación**
 - ▶ Ej.: Documentación no acorde con lo que hace el software
- ▶ **De sobrecarga**
 - ▶ Ej.: El sistema funciona bien con 100 usuarios pero no con 110.

Tipos de Defectos

- ▶ De capacidad
 - ▶ Ej.: El sistema funciona bien con ventas $< 1.000.000$
- ▶ De coordinación o sincronización
 - ▶ Ej.: Comunicación entre procesos con fallas
- ▶ De rendimiento
 - ▶ Ej.: Tiempo de respuesta inadecuado.
- ▶ De recuperación
 - ▶ Ej.: No volver a un estado normal luego de una falla
- ▶ De relación hardware-software
 - ▶ Ej.: Incompatibilidad entre componentes
- ▶ De estándares
 - ▶ Ej.: No cumplir con la definición de estándares y procedimientos

Clasificación ortogonal de Defectos

- ▶ Los defectos se han organizado en categorías
- ▶ Primeramente se debe identificar si es un:
 - ▶ Defecto por omisión o Defecto de cometido

Tipo de defecto	Significado
Función	Afecta la capacidad, interfaces.
Interfaz	Afecta a la interacción con otros componentes.
Comprobación	Afecta la lógica del programa.
Asignación	Afecta la estructura de datos.
Sincronización	Involucra sincronización de recursos compartidos y de tiempo real.
Construcción	Ocurre debido a problemas en repositorios, gestión de cambios o control de versiones.
Documentación	Afecta a publicaciones.
Algoritmo	Involucra la eficiencia o exactitud de un algoritmo.

Objetivos y Beneficios de las Pruebas del Software

- ▶ Descubrir errores antes que el software salga del ambiente de desarrollo
- ▶ Detectar un error no descubierto hasta entonces.
- ▶ Bajar los costos de corrección de errores en la etapa de mantenimiento

Principios de la Prueba

- ▶ A todas las pruebas se les debería poder hacer un seguimiento hasta los requisitos del cliente.
- ▶ Las pruebas deberían planificarse mucho antes de que empiecen.
- ▶ Las pruebas deberían empezar por «lo pequeño» y progresar hacia «lo grande »
- ▶ Para ser más eficaces, las pruebas deberían ser realizadas por un equipo independiente.

¿Quién realiza las pruebas?

- ▶ Varios factores justifican un equipo independiente de pruebas, entre ellos:
 - ▶ Evitar el conflicto entre la responsabilidad por los defectos y la necesidad de descubrir defectos
 - ▶ Llevar a cabo las pruebas concurrentemente con la codificación.
 - ▶ Los desarrolladores deben colaborar y corregir los errores.

Pruebas del Software

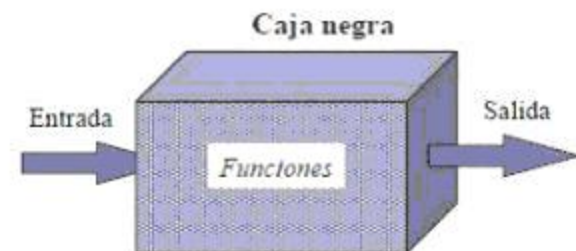
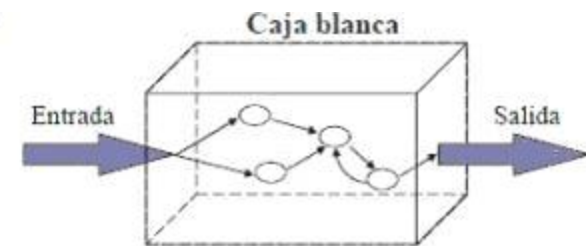
- ▶ Lamentablemente “La prueba NO puede asegurar la ausencia de defectos”
- ▶ Se intentan buscar Casos de Prueba que permitan encontrar errores

Pruebas del Software

- ▶ **Cualquier software puede probarse de una de estas dos formas:**
 - ▶ Conociendo la función específica para el que fue diseñado, se pueden llevar a cabo pruebas que demuestren que cada función es completamente operativa, y al mismo tiempo buscando errores en cada función.
 - ▶ Conociendo el funcionamiento del producto, se pueden desarrollar pruebas que aseguren que todas las piezas encajan, o sea que la operación interna se ajusta a las especificaciones y que todos los componentes internos se han comprobado de forma adecuada.
- ▶ **El primer enfoque se denomina prueba de caja negra y el segundo, prueba de caja blanca**

Tipos de Pruebas del Software

- ▶ La prueba de caja negra se refiere a las pruebas que se llevan a cabo sobre la interfaz del software.
- ▶ La prueba de caja blanca se basa en el minucioso examen de los detalles procedimentales. Se comprueban los caminos lógicos del software proponiendo casos de prueba que ejerciten conjuntos específicos de condiciones y/o bucles.



Tipos de Pruebas

Caja Negra (o Cerrada)

- ▶ También denominada prueba de comportamiento, se centran en los requisitos funcionales del software.
- ▶ Intenta descubrir diferentes tipos de errores que los métodos de caja blanca.

Tipos de Pruebas

Caja Negra (o Cerrada)

- ▶ ¿Qué errores busca?
 - ▶ Funciones incorrectas o ausentes
 - ▶ Errores de interfaz
 - ▶ Errores en estructuras de datos o en accesos a bases de datos externas
 - ▶ Errores de rendimiento
 - ▶ Errores de inicialización y de terminación
-
- ▶ Concentra la prueba en el dominio de información: por ejemplo las pruebas de partición equivalente

Prueba De Partición Equivalente

- ▶ El diseño de los casos de prueba se basa en una evaluación de las clases de equivalencia para una condición de entrada.
- ▶ Una clase de equivalencia representa un conjunto de estados válidos o no válidos para condiciones de entrada.
- ▶ Una condición de entrada es un valor numérico específico, un rango de valores, un conjunto de valores relacionados o una condición lógica.

Prueba De Partición Equivalente

- ▶ Las clases de equivalencia se pueden definir de acuerdo con las siguientes directrices:
 - ▶ Si una condición de entrada especifica un rango, se define una clase de equivalencia válida y dos no válidas.
 - ▶ Si una condición de entrada requiere un valor específico, se define una clase de equivalencia válida y dos no válidas.
 - ▶ Si una condición de entrada especifica un elemento de un conjunto, se define una clase de equivalencia válida y una no válida.
 - ▶ Si una condición de entrada es lógica, se define una clase de equivalencia válida y una no válida.

Prueba De Partición Equivalente Ejemplo

DAR DE ALTA JUGUETE :

Código:

Nombre:

Descripción:

Recomendaciones:

Género:

Edad:

Marca:

Stock: Stock mínimo:

Estado:

Definición de tipos de datos

▶ DATO	TIPO
▶ Código	entero positivo
▶ Nombre	string 20
▶ descripción	string 256
▶ Recomendaciones	string 512
▶ Genero	enumerativo (masculino, femenino)
▶ Edad	rango 0..120
▶ Marca	string 25
▶ Stock	entero
▶ stock mínimo	entero positivo
▶ Estado	enumerativo (normal, oferta, novedoso)

Identificación de las clases de equivalencia.

Condición de entrada	Clases de equivalencia válidas	Clases de equivalencia inválidas
código	0<= código <= 9999	Código <0 Código >10000
nombre	1 a 20 caracteres	0 caracteres; mas de 20 caracteres;
descripción	0 a 256 caracteres	mas de 256 caracteres
recomendaciones	0 a 512 caracteres	mas de 512 caracteres
genero	masculino femenino	otra cadena de caracteres; número
stock	Número entero	Caracteres no dígitos; Número no entero

Análisis de Valores Límites (AVL)

- ▶ Los errores tienden a darse más en los límites del campo de entrada que en el «centro».
- ▶ Por ello, se ha desarrollado el análisis de valores límites (AVL) como técnica de prueba.
- ▶ Complementa a la partición equivalente. En lugar de seleccionar cualquier elemento de una clase de equivalencia, el AVL selecciona los casos de prueba en los «extremos» de la clase. En lugar de centrarse solamente en las condiciones de entrada, el AVL obtiene casos de prueba también para el campo de salida

Análisis de Valores Límites (AVL)

- ▶ Casos de prueba en los bordes de las clases:
- ▶ Para una condición de entrada de rango entre a y b
probar: a, b, $\leq a$ y $\geq b$.
- * Para una salida de rango entre a y b: utilizar casos de prueba que generen valor de salida a y b
- * Probar las estructuras de datos internas en sus límites.

Tipos de Pruebas

Caja Blanca (o Cristal o Abierta)

- ▶ Deriva casos de prueba de la estructura de control, para verificar detalles procedimentales. Mediante los métodos de prueba de caja blanca, el ingeniero del software puede obtener casos de prueba que ..
 - ▶ garanticen que se ejercita por lo menos una vez todos los caminos independientes de cada módulo.
 - ▶ ejerciten todas las decisiones lógicas .
 - ▶ ejecuten todos los bucles en sus límites..
 - ▶ ejerciten las estructuras internas de datos para asegurar su validez.

Tipos de Pruebas

Caja Blanca (o Cristal o Abierta)

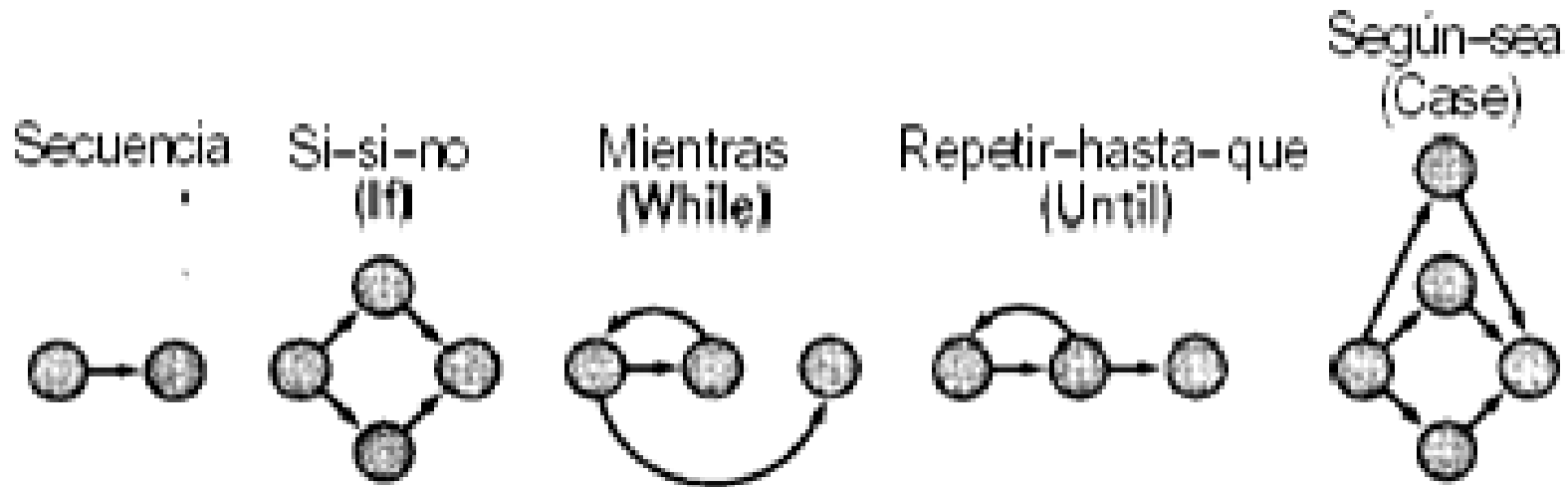
- ▶ El flujo lógico de un programa a veces no es nada intuitivo, lo que significa que nuestras suposiciones intuitivas sobre el flujo de control y los datos nos pueden llevar a tener errores de diseño que sólo se descubren cuando comienza la prueba del camino.
- ▶ ¿Por qué realizarlas?
- ▶ Los casos especiales son los más factibles de error
- ▶ Los errores tipográficos son aleatorios
- ▶ El flujo de control intuitivo es distinto del real

Prueba Del Camino Básico

- ▶ Es una técnica propuesta por Tom McCabe. Permite al diseñador de casos de pruebas obtener una medida de la complejidad lógica y usarla como guía para la definición de caminos de ejecución.
- ▶ Los casos de prueba obtenidos garantizan que se ejecuta al menos una vez cada sentencia del programa.

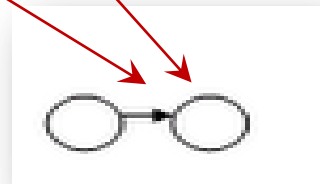
Prueba Del Camino Básico

Notación de grafo de flujo :



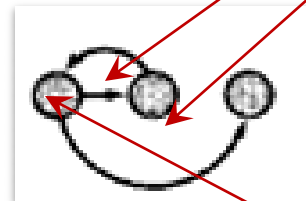
Prueba Del Camino Básico

- ▶ Cada círculo, denominado nodo del grafo de flujo, representa una o más sentencias procedimentales.
- ▶ Un solo nodo puede corresponder a una secuencia de acciones y a una decisión. Las flechas del grafo de flujo, denominadas aristas o enlaces, representan flujo de control.
- ▶ Una arista debe terminar en un nodo.



Prueba Del Camino Básico

- ▶ Las áreas delimitadas por aristas y nodos se denominan regiones. Cuando contabilizamos las regiones incluimos el área exterior del grafo, contándolo como otra región más.



- ▶ Cada nodo que contiene una condición se denomina nodo predicado y está caracterizado porque dos o más aristas emergen de él.

Prueba Del Camino Básico

- ▶ La complejidad ciclomática es una métrica del software que proporciona una medición cuantitativa de la complejidad lógica de un programa.
- ▶ La complejidad ciclomática define el número de caminos independientes del conjunto básico de un programa y nos da un límite superior para el número de pruebas que se deben realizar para asegurar que se ejecuta cada sentencia al menos una vez.
- ▶ Un camino independiente es cualquier camino del programa que introduce, por lo menos, un nuevo conjunto de sentencias de proceso o una nueva condición.

Prueba Del Camino Básico

► Complejidad ciclomática :

- 1. $V(g)$ = Cantidad de regiones del grafo ó
- 2. $V(g) = A - N + 2$ ó
- 3. $V(g) = P + 1$

► La complejidad ciclomática debe medirse con las tres formulas de manera de verificar su exactitud.

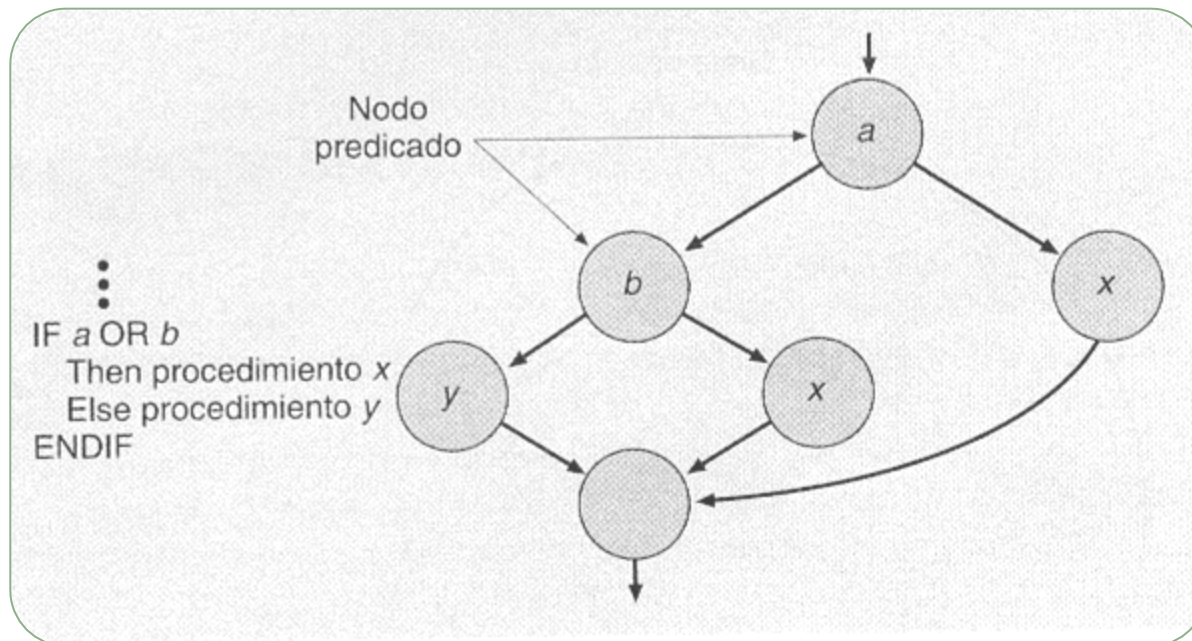
Prueba Del Camino Básico

► Pasos :

- Dibujar el grafo de flujo correspondiente.
- Determinar la complejidad ciclomática.
- Determinar un conjunto básico de caminos independientes.
- Preparar los casos de prueba que forzarán la ejecución de cada camino del conjunto.
- Ejecutar cada caso de prueba y comparar los resultados obtenidos con los esperados.

Prueba Del Camino Básico

► Condiciones compuestas: OR,AND



Prueba Del Camino Básico

LPD

Procedimiento Ordenar

1 do while queden registros

 leer registros

2 if campo 1 de registro = 0

3 then procesar registro

 Guardar en buffer

 Incrementar contador

4 else if campo 2 de registro = 0

5 then reiniciar contador

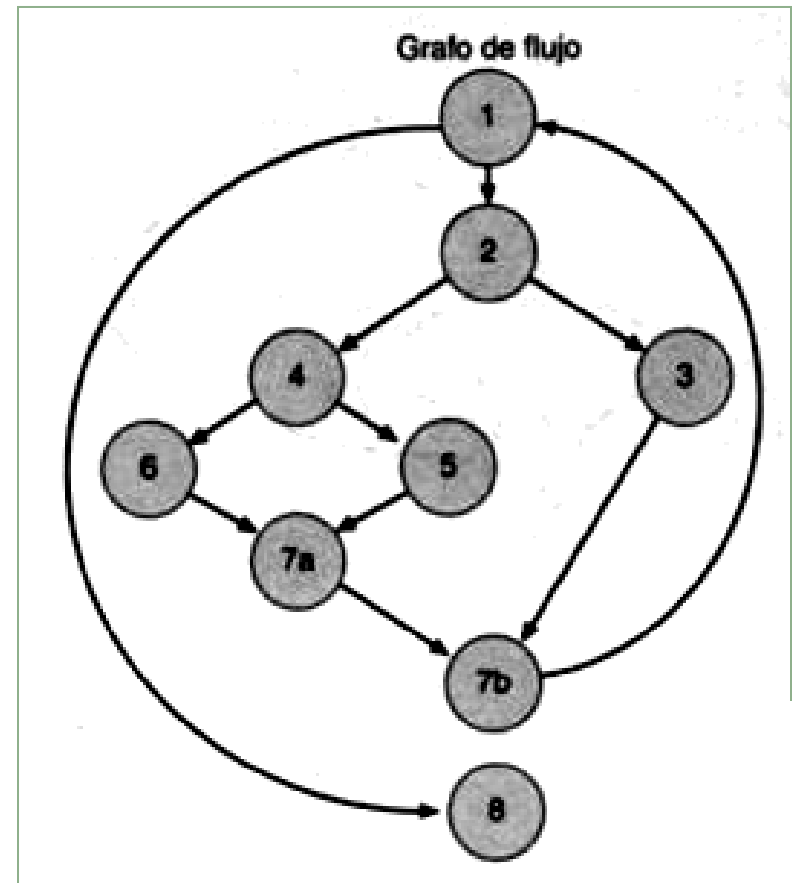
6 else procesar registro

 Guardar en archivo

7a endif

7b endif

8 enddo



Prueba Del Camino Básico

PROCEDURE media;

- Este procedimiento calcula la media de 100 o menos números que se encuentren entre unos límites; también calcula el total de entradas y el total de números válidos.

INTERFACE RETURNS media, total, entrada, total.válido;

INTERFACE ACCEPTS valor, minimo, máximo;

TYPE valor [1:100] IS SCALAR ARRAY;

TYPE media, total, entrada, total, válido;

Minimo, máximo, suma IS SCALAR;

TYPE i IS INTEGER;

i = 1;

total, entrada = total, válido = 0;

suma = 0;

DO WHILE **VALOR [i] < -999** and **total.entrada < 100**

Incrementar total.entrada en 1;

IF **valor [i] >= minimo** AND **valor [i] <= máximo**

THEN incrementar total.válido en 1;

suma = suma + valor [i];

ELSE ignorar

ENDIF

Incrementar i en 1;

ENDDO

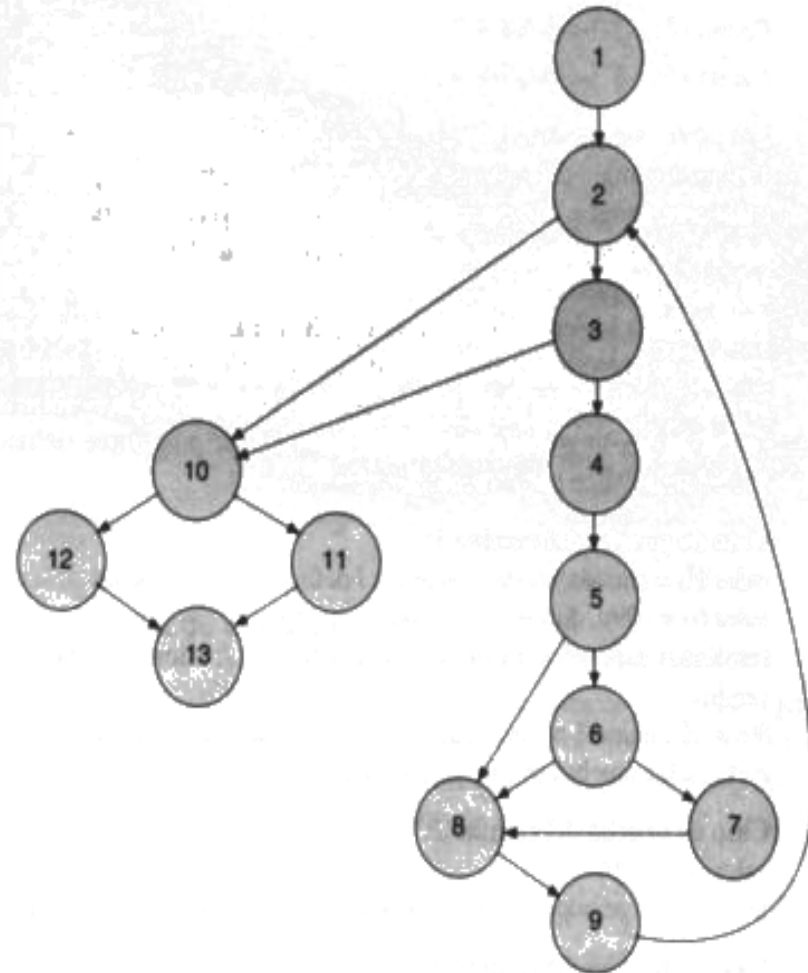
If **total.válido > 0**

THEN media = suma/total.válido;

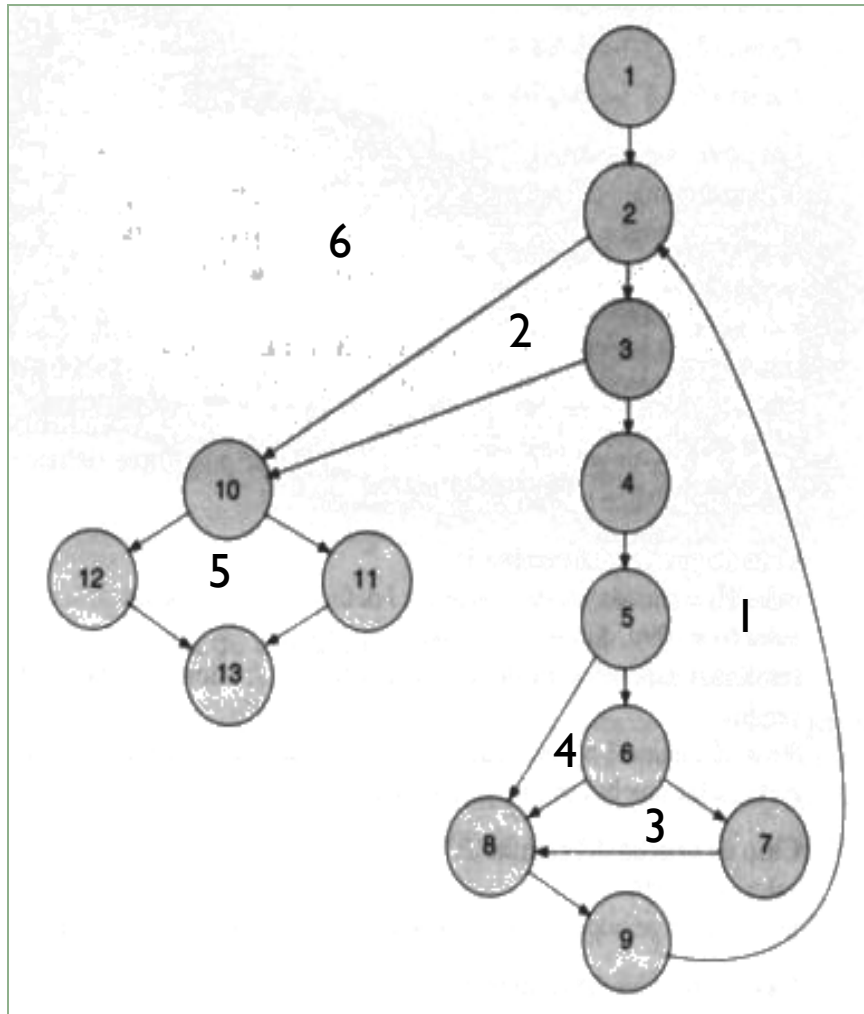
ELSE media = -999;

ENDIF

END media



Prueba Del Camino Básico



Regiones : 6

Nodos : 13

Aristas : 17

Nodos Predicados : 5

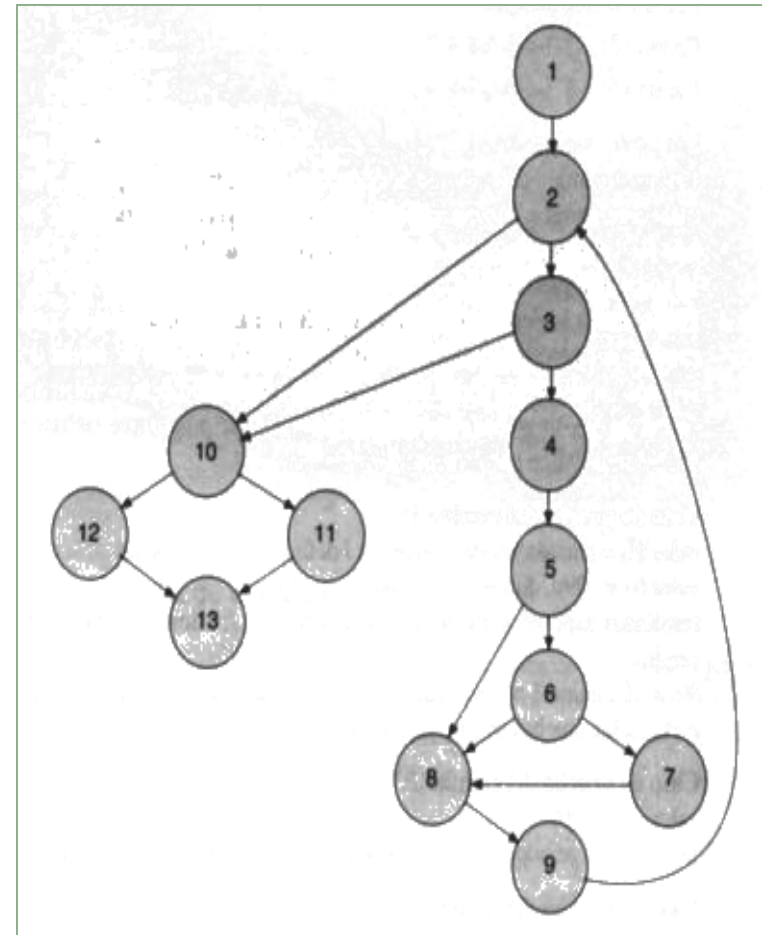
$$V(G) : A - N + 2 = 17 - 13 + 2 = 6$$

$$V(G) : NP + 1 = 5 + 1 = 6$$

$$V(G) : R = 6$$

Prueba Del Camino Básico

- ▶ Camino 1: 1-2-10-11-13
- ▶ Camino 2: 1-2-10-12-13
- ▶ Camino 3: 1-2-3-10-11-13
- ▶ Camino 4: 1-2-3-4-5-8-9-2...
- ▶ Camino 5: 1-2-3-4-5-6-8-9-2...
- ▶ Camino 6: 1-2-3-4-5-6-7-8-9-2...



Prueba Del Camino Básico

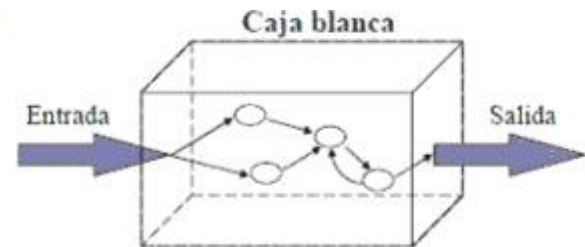
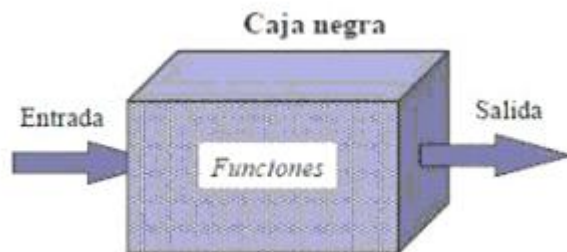
- ▶ **Caso de prueba del camino 2:**
 - ▶ $\text{valor}(l) = -999$
 - ▶ resultados esperados: media = -999, otros totales con sus valores iniciales

Prueba Del Camino Básico

- ▶ **Caso de prueba del camino 3:**
 - ▶ intento de procesar 101 o más valores
 - ▶ los primeros 100 valores deben ser válidos
 - ▶ resultados esperados: ídem camino 1

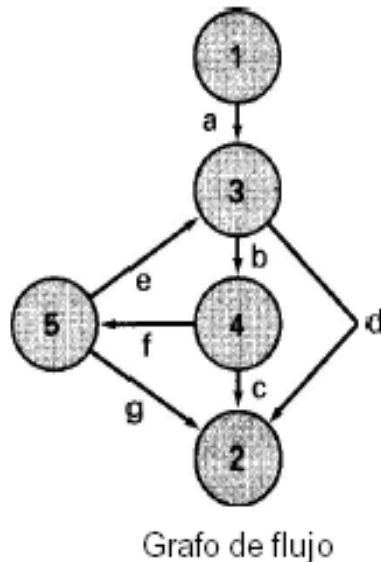
Pruebas del Software

- ▶ La prueba de caja negra se refiere a las pruebas que se llevan a cabo sobre la interfaz del software.
- ▶ La prueba de caja blanca se basa en el minucioso examen de los detalles procedimentales. Se comprueban los caminos lógicos del software proponiendo casos de prueba que ejerciten conjuntos específicos de condiciones y/o bucles.



Prueba de caja Blanca. Matriz de Grafos

- Se puede automatizar la prueba del camino básico, generando una Matriz de Grafos



		Conectado al				
Nodo		Nodo				
Nodo		1	2	3	4	5
1				a		
2						
3			d		b	
4			c			f
5			g	e		

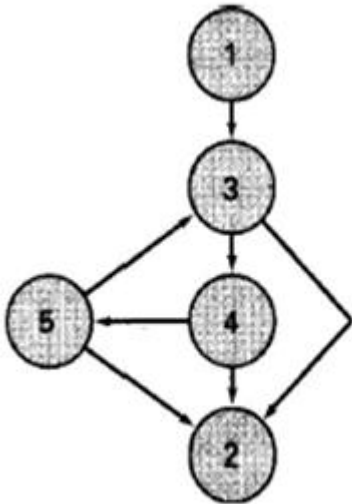
Matriz de Grafos

- ▶ Añadiendo un peso de enlace a cada entrada de la matriz, contamos con mas información
- ▶ El peso es 1 (existe conexión) 0 (no la hay)
- ▶ También se puede asignar prob, tiempo, mem requerida, recursos usados etc.
- ▶ Con ella contamos una manera mas de obtener la Complejidad Ciclomática

Matriz de Grafos



Cada fila con dos o mas entradas representa un nodo predicado



	1	2	3	4	5
1			1		
2					
3		1		1	
4		1			1
5		1	1		

nodos predicados

$$V(G) = 3 + 1 = 4$$



Grafos Causa - Efecto

- ▶ Ayudan a explorar combinaciones de circunstancias de entradas
 - ▶ Listar para cada módulo entradas y acciones
 - ▶ Desarrollar el grafo
 - ▶ Convertir el grafo a una tabla de decisión
 - ▶ Convertir las reglas en casos de prueba

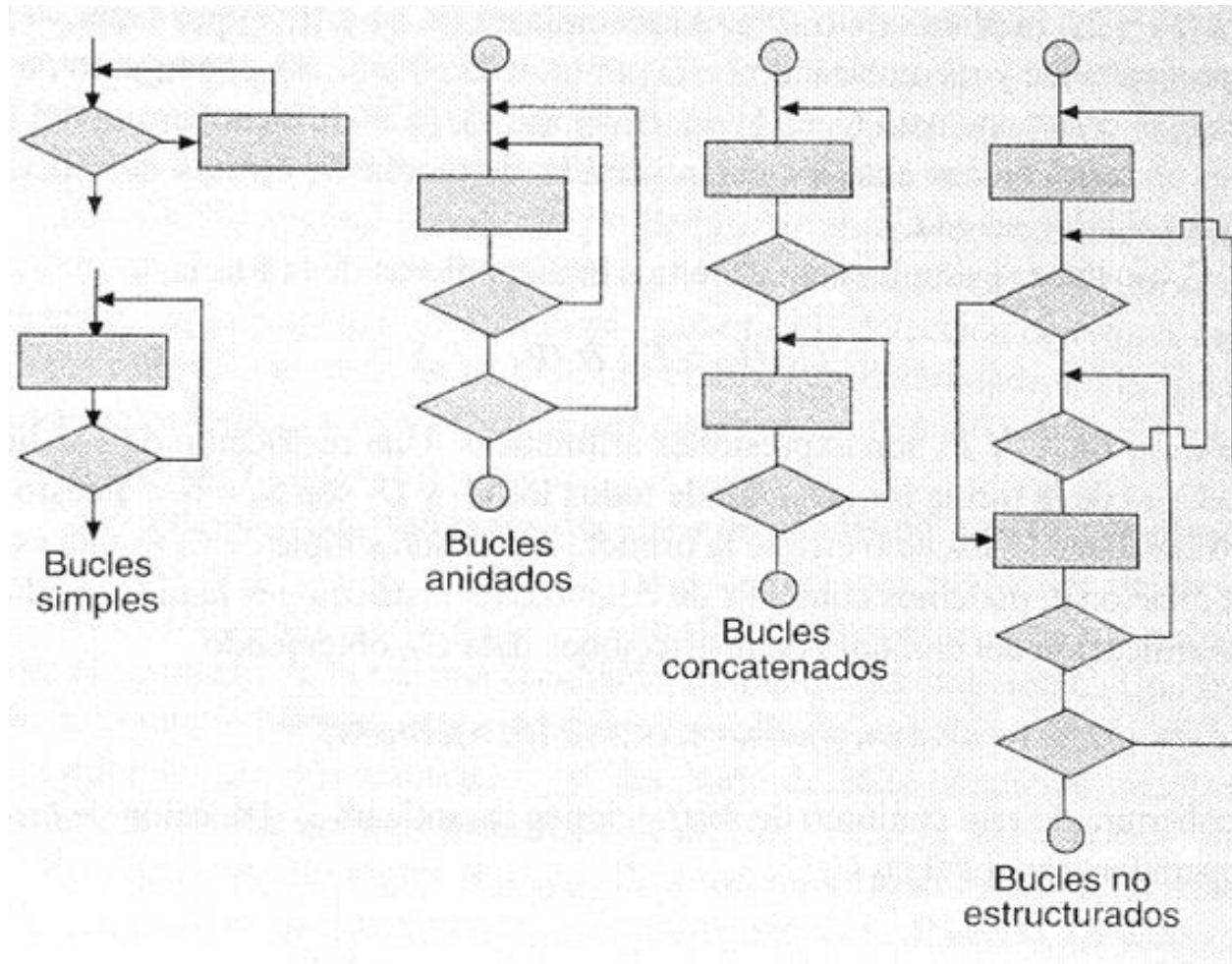
Pruebas de Bucles

La prueba de bucles es una técnica de prueba de caja blanca que se centra exclusivamente en la validez de las construcciones de bucles.

Se pueden definir cuatro clases diferentes de bucles

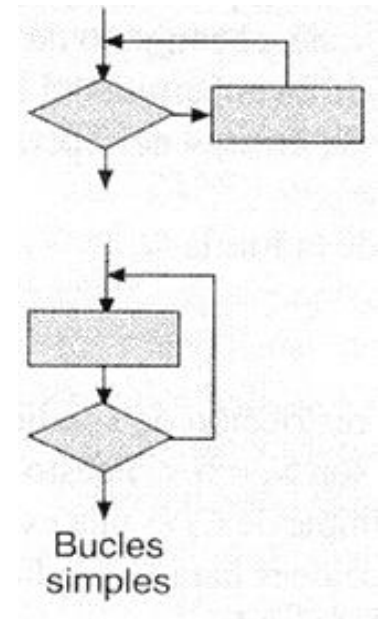
- *Bucles simples,*
- *Bucles concatenados,*
- *Bucles anidados y*
- *Bucles no estructurados*

Pruebas de Bucles



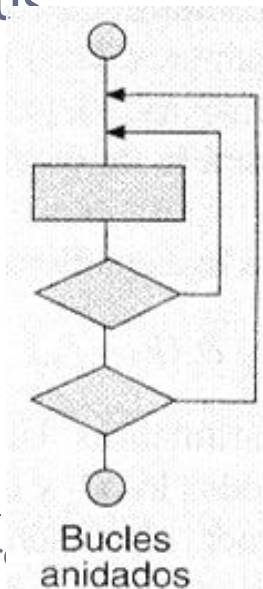
Pruebas de Bucles

- ▶ **Bucles simples:** Se les debe aplicar el siguiente conjunto de pruebas, donde n es el número máximo de pasos permitidos por el bucle:
 - ▶ 1. pasar por alto totalmente el bucle
 - ▶ 2. pasar una sola vez por el bucle
 - ▶ 3. pasar dos veces por el bucle
 - ▶ 4. hacer m pasos por el bucle con $m < n$
 - ▶ 5. hacer $n - 1$, n y $n + 1$ pasos por el bucle



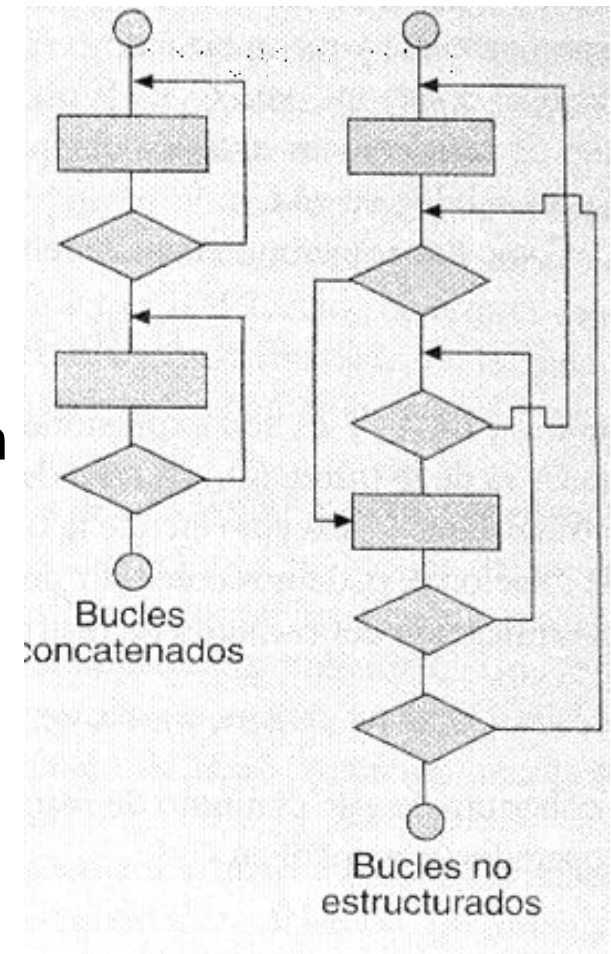
Pruebas de Bucles

- ▶ **Bucles anidados:** Se sugiere un enfoque que ayuda a reducir el número de pruebas:
 1. Comenzar por el bucle más interior. Establecer o configurar los demás bucles con sus valores mínimos.
 - 2..Añadir otras pruebas para valores fuera de rango o excluidos.
 3. Progresar hacia fuera, llevando a cabo pruebas para el siguiente bucle, pero manteniendo todos los bucles externos en sus valores mínimos y los demás bucles anidados en sus valores «típicos».
 4. Continuar hasta que se hayan probado todos los bucles.



Pruebas de Bucles

- ▶ Bucles concatenados : Si hay dos bucles concatenados y no son independientes, usar el enfoque aplicado para los bucles anidados.
- ▶ Los bucles no estructurados, se deben rediseñar



Prueba de entornos especializados

A medida que el software se hace mas complejo, ha crecido también la necesidad de enfoques de pruebas especializados.

Los métodos de prueba de caja blanca y negra son aplicables a todos los entornos, pero a veces se necesitan un enfoque único.

Prueba de arquitectura cliente/servidor

Prueba de servidor: probar las funciones de coordinación y manejo de datos del servidor.

Desempeño del servidor (tiempo de respuesta y procesamiento total de los datos)

Prueba de base de datos: probar la exactitud e integridad de los datos, examinar transacciones, asegurar que se almacena, actualiza y recuperan los datos.

Pruebas de comunicación de red: verificar comunicación entre los nodos, el paso de mensajes, transacciones y trafico de la red se realice sin errores.

Prueba de la Documentación

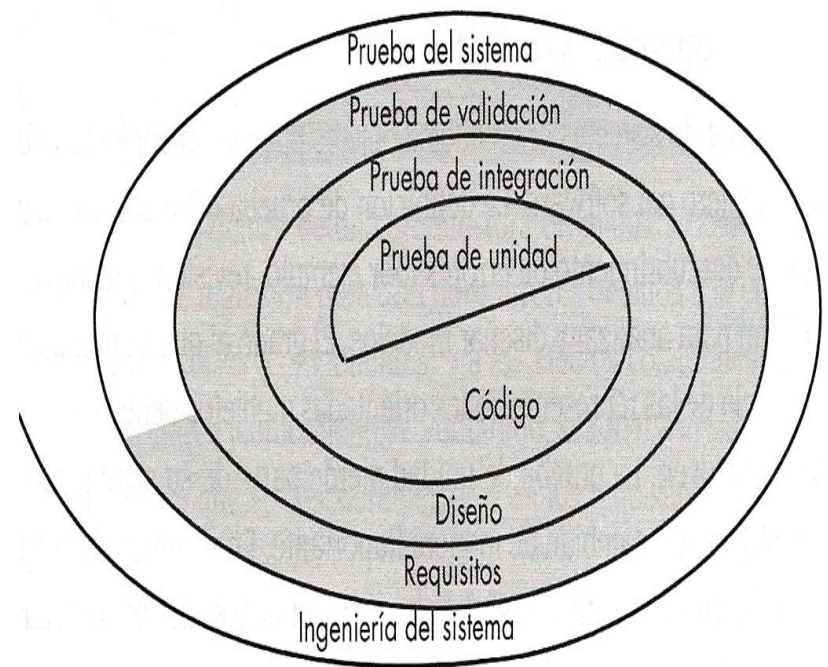
- ▶ Es importante para la aceptación del programa.
 - ▶ Revisar la guía de usuario o funciones de ayuda en línea.
 - ▶ Prueba de documentación es en dos fases:
 - ▶ 1. Revisar e inspeccionar: examinar la claridad editorial del documento.
 - ▶ 2. Prueba en vivo: usar la documentación junto con el programa real.

Estrategia de Prueba

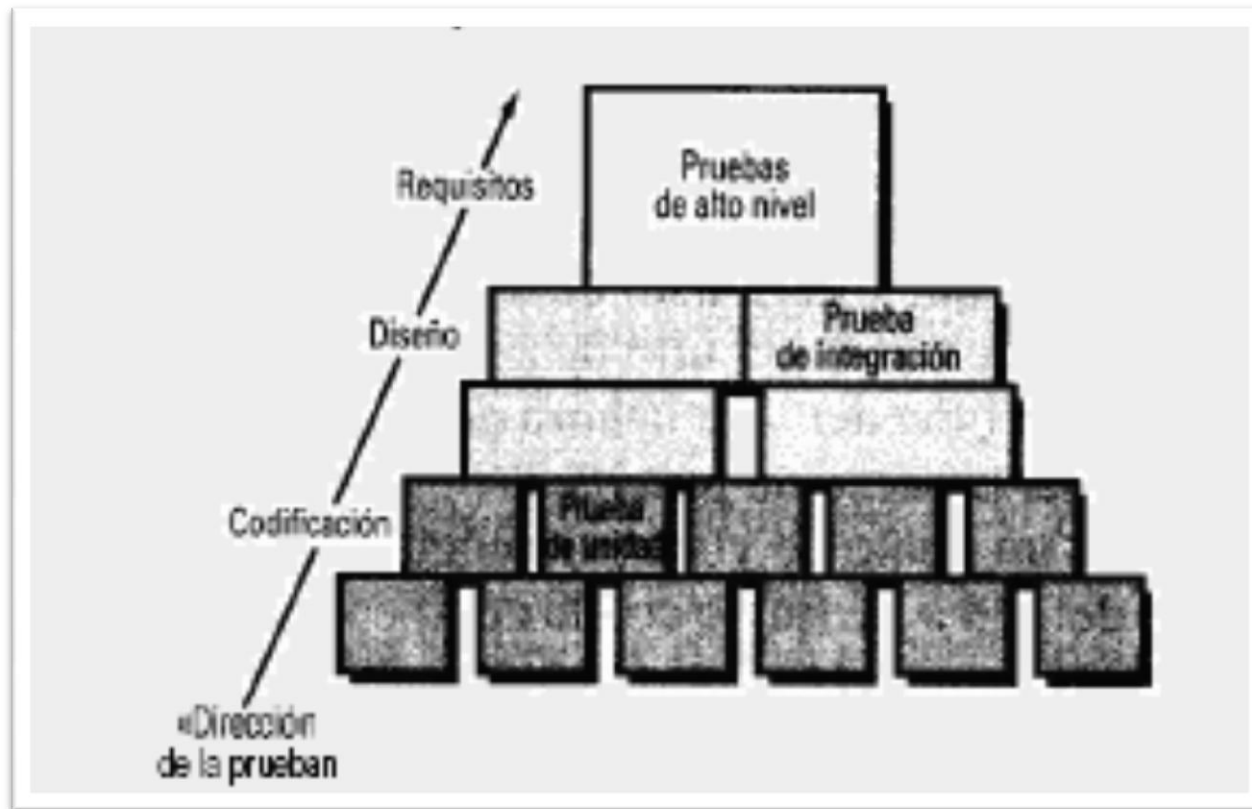
- ▶ Las pruebas comienzan a nivel de módulo y trabajan «hacia fuera», hacia la integración de todo el sistema.
- ▶ Según el momento, son apropiadas diferentes técnicas de prueba.
- ▶ La prueba y la depuración son actividades diferentes, pero la depuración se debe incluir en cualquier estrategia de prueba.
- ▶ Una estrategia de prueba del software debe incluir pruebas de bajo nivel que, así como pruebas de alto nivel, validen las principales funciones del sistema frente a los requisitos del cliente.
- ▶ Una estrategia debe proporcionar una guía.

Estrategia de Prueba

- ✓ La estrategia de prueba del software es un proceso en espiral
- ✓ La prueba de unidad comienza la espiral.
- ✓ Al movernos hacia fuera de la espiral, se llega a la prueba de integración, donde el foco de atención es el diseño y la arquitectura del software.
- ✓ Otra vuelta por la espiral hacia fuera, encontramos la *prueba de validación*, donde se validan los requisitos establecidos
- ✓ Finalmente, llegamos a la *prueba del sistema*, en la que se prueban el software como un todo.
- ✓ La *prueba del sistema* verifica que cada elemento encaja de forma adecuada y que se alcanza la funcionalidad y el rendimiento del sistema total.



Estrategia de Prueba- Pasos

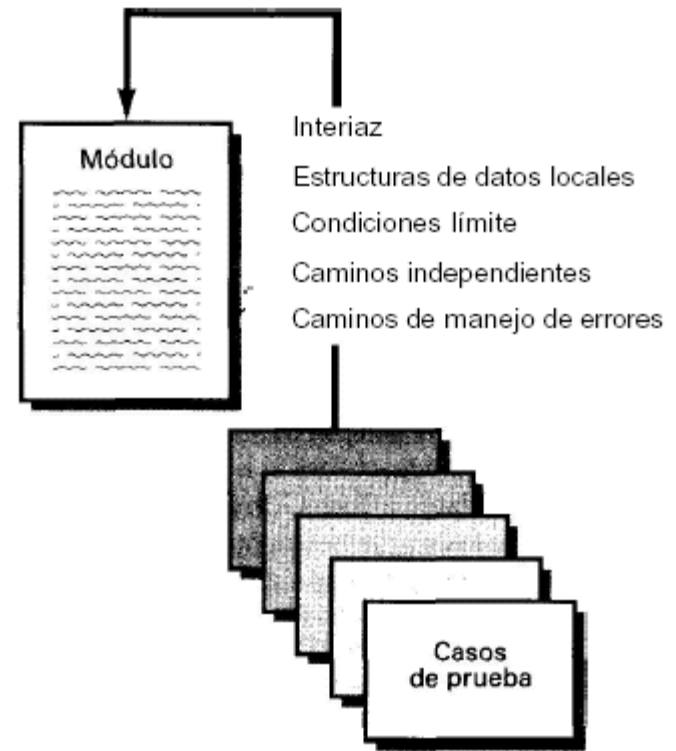


Tipos de Pruebas

- ▶ Las pruebas de unidad verifican que el componente funciona correctamente a partir del ingreso de distintos casos de prueba. Se realiza utilizando técnicas de prueba de caja blanca y caja negra.
- ▶ Las pruebas de integración verifican que los componentes trabajan correctamente en forma conjunta. Durante la integración, las técnicas mas utilizadas son las de prueba de caja negra.
- ▶ Las pruebas de validación proporcionan una seguridad final de que el software satisface todos los requisitos funcionales y no funcionales.. Durante la validación se usan exclusivamente técnicas de caja negra.
- ▶ La prueba del sistema verifica que cada elemento encaja de forma adecuada y que se alcanza la funcionalidad y el rendimiento del sistema total.

Pruebas de Unidad

- ✓ Se prueba la interfaz del módulo para asegurar que la información fluye de forma adecuada.
- ✓ Se examinan las estructuras de datos locales.
- ✓ Se prueban las condiciones límite para asegurar que el módulo funciona correctamente
- ✓ Se ejercitan todos los caminos independientes.
- ✓ Las pruebas del camino básico y de bucles son las técnicas mas usadas.



Que Probar ?

▶ ➔ MODULOS CRITICOS

- Dirigidos a varios requerimientos
- Con alto nivel de control
- Complejos
- Con requerimientos de rendimiento muy definidos

Pruebas de Integración

- ▶ Los datos se pueden perder en una interfaz.
- ▶ Un módulo puede tener un efecto adverso e inadvertido sobre otro etc..
- ▶ Entonces...
 - ▶ La Técnica para construir la estructura del programa y para detectar errores de interacción es la Prueba de Integración.

- ▶ Tipos :
 - INTEGRACION INCREMENTAL
 - NO INCREMENTAL (BIG BANG)

Pruebas de Integración

Integración incremental

- ▶ El programa se construye y se prueba en pequeños segmentos en los que los errores son más fáciles de aislar y de corregir
- ▶ La Integración Incremental puede ser :
 - ▶ Descendente (Top-Down)
 - ▶ Ascendente (Bottom-Up)
 - ▶ Mezcla

Integración Descendente

► Se puede realizar :

- En profundidad : primero-en-profundidad integra todos los módulos de un camino de control principal de la estructura.

Pasos:

1. Conductor: Módulo principal

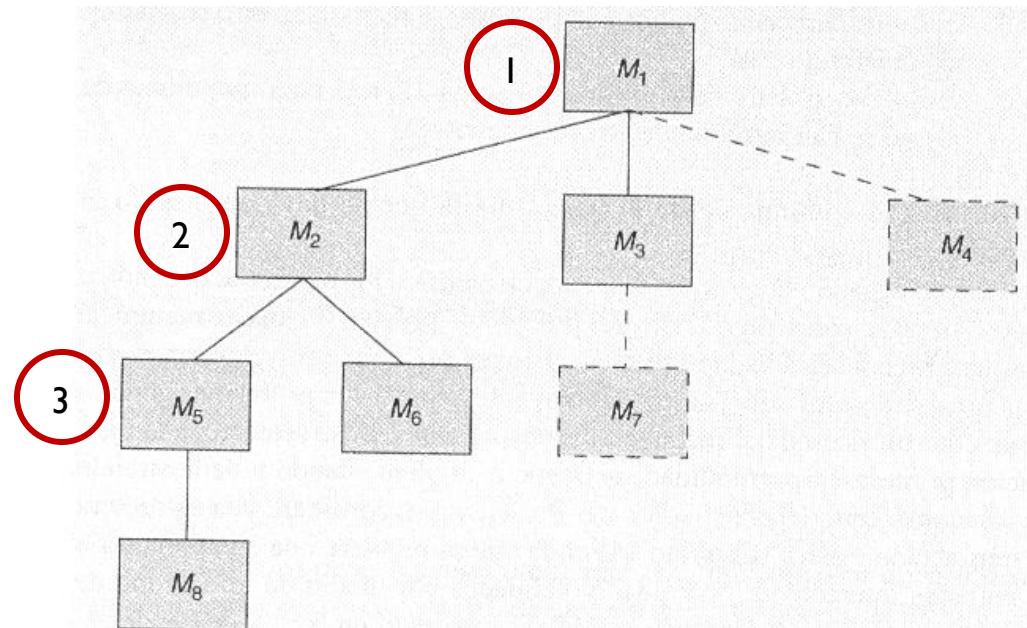
Resguardos: Para los módulos subordinados

2. Sustituir resguardos por módulos I a I

3. Probar

4. Reemplazar otro resguardo

5. Pruebas de regresión



Integración Descendente

► Se puede realizar :

- En anchura : primero-en-anchura incorpora todos los módulos directamente subordinados a cada nivel.

Pasos:

1. Conductor: Módulo principal

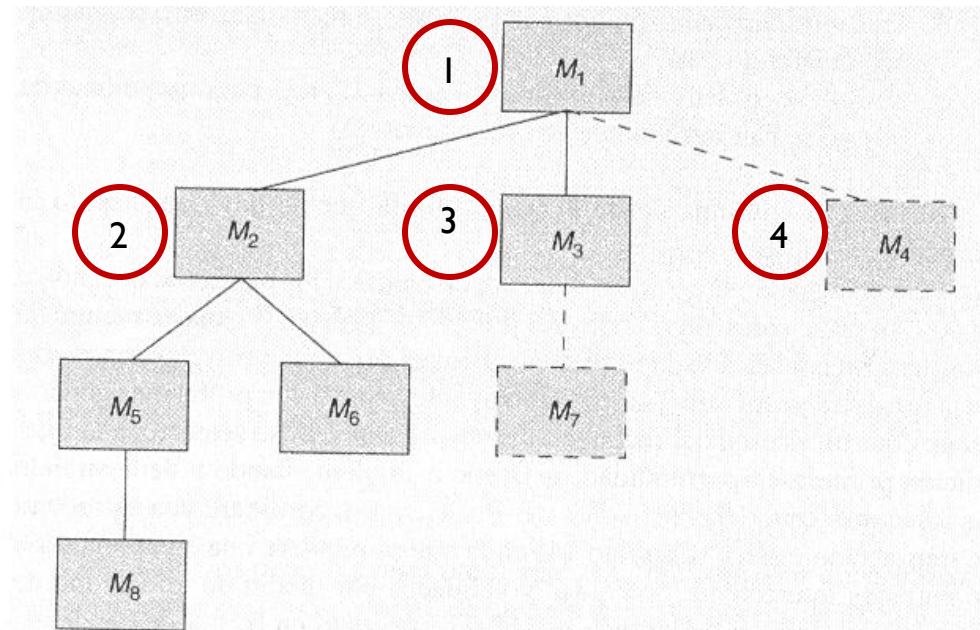
Resguardos: Para los módulos subordinados

2. Sustituir resguardos por módulos 1 a 1

3. Probar

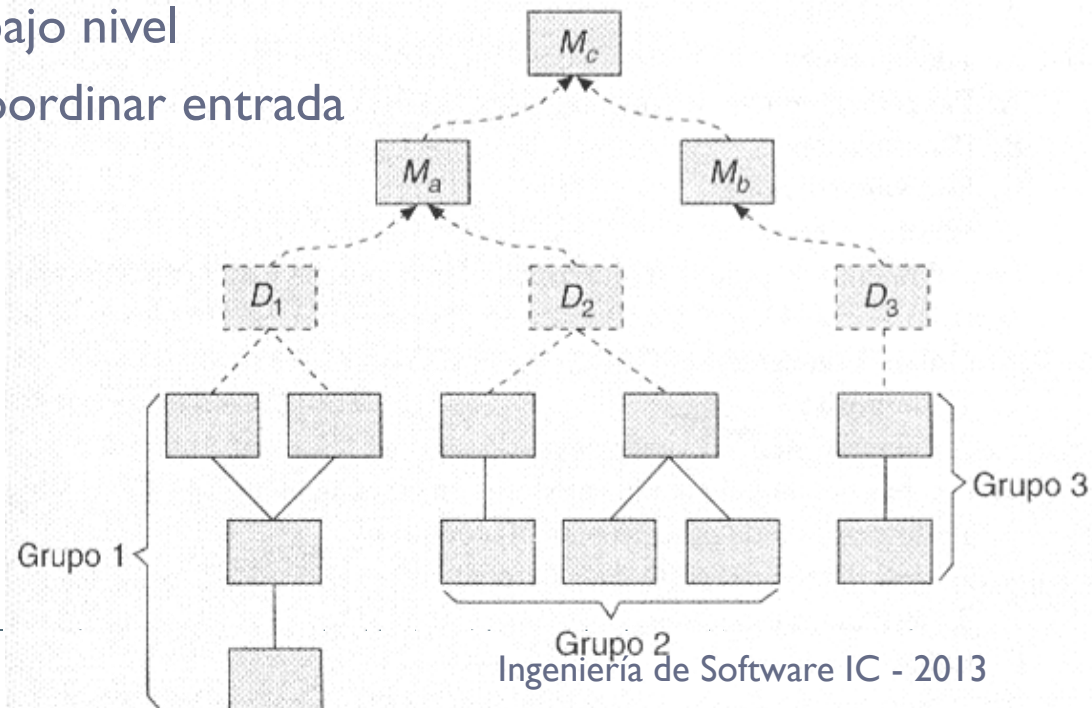
4. Reemplazar otro resguardo

5. Pruebas de regresión



Integración Ascendente

- ▶ Se empieza la prueba con los módulos atómicos (es decir, módulos de los niveles más bajos de la estructura del programa).
- ▶ Dado que los módulos se integran de abajo hacia arriba, el proceso requerido de los módulos subordinados siempre está disponible y se elimina la necesidad de resguardos pero no así, los conductores.
- ▶ Pasos :
 - ▶ 1. Combinar módulos de bajo nivel
 - ▶ 2. Hacer conductor para coordinar entrada y salida
 - ▶ 3. Probar el grupo
 - ▶ 4. Eliminar conductores



Pruebas de regresión

- ▶ Cada vez que se añade un nuevo módulo como parte de una Prueba de integración, el software cambia.
- ▶ Se establecen nuevos caminos, pueden ocurrir nuevas E/S y se invoca una nueva lógica de control.
- ▶ Estos cambios pueden causar problemas con funciones que antes trabajaban perfectamente.
- ▶ En el contexto de una estrategia de Prueba de integración, la Prueba de regresión es volver a ejecutar un subconjunto de pruebas que se han llevado a cabo anteriormente para asegurarse de que los cambios no han propagado efectos colaterales no deseados.

Pruebas de regresión

- ▶ Estas pruebas se puede hacer manualmente, volviendo a realizar un subconjunto de todos los casos de prueba o utilizando herramientas automáticas
- ▶ El conjunto de pruebas de regresión contiene tres clases diferentes de casos de prueba:
 - ▶ una muestra representativa de pruebas que ejercite todas las funciones del software
 - ▶ pruebas adicionales que se centren en las funciones del software que son probablemente afectadas por el cambio
 - ▶ pruebas que se centren en los componentes del software que han cambiado.

Pruebas de Validación

- ▶ La validación del software se consigue mediante una serie de pruebas de caja negra que demuestran la conformidad con los requisitos.
- ▶ Una vez que se procede con cada caso de prueba de validación, puede darse una de las dos condiciones
 - ▶ (1) las características de funcionamiento o de rendimiento están de acuerdo con las especificaciones y son aceptables; o
 - ▶ (2) se descubre una desviación de las especificaciones y se crea una lista de deficiencias.

Pruebas de Validación

- ▶ En síntesis las pruebas de “validación” en general utilizan técnicas de caja negra para validar los requerimientos funcionales, el rendimiento, la documentación, la portabilidad, la compatibilidad, la recuperación de errores, la mantenibilidad, la seguridad, la resistencia.

Pruebas de Aceptación

- ▶ Cuando se construye software a medida para un cliente, se llevan a cabo una serie de pruebas de aceptación para permitir que el cliente valide todos los requisitos.
- ▶ Las realiza el usuario final en lugar del responsable del desarrollo del sistema, una prueba de aceptación puede ir desde algo informal, hasta la ejecución sistemática de una serie de pruebas bien planificadas.
- ▶ Dentro de las Pruebas de aceptación se pueden encontrar :
 - ▶ Pruebas ALFA: desarrolladores con clientes antes de liberar el producto.
 - ▶ Pruebas BETA: seleccionando los clientes que efectuarán la prueba. El desarrollador no se encuentra presente.

Pruebas del Sistema

- ▶ La prueba del sistema, está constituida por una serie de pruebas diferentes.
- ▶ Aunque cada prueba tiene un propósito diferente, todas trabajan para verificar que se han integrado adecuadamente todos los elementos del sistema y que realizan las funciones apropiadas.

Pruebas del Sistema

► Tipos de Pruebas del Sistema :

- Pruebas de recuperación : Se controla la recuperación de fallas y el modo de reanudación del procesamiento en un tiempo determinado. Generalmente se fuerza el fallo para comprobarlo.
- Pruebas de seguridad : Se comprueban los mecanismos de protección integrados.
- Pruebas de resistencia (Stress) : Se diseñan para enfrentar a los programas a situaciones anormales.
- Prueba de rendimiento : Se prueba el sistema en tiempo de ejecución. A veces va emparejada con la Prueba de resistencia.